

OOP-2. 객체의 종류

객체의 종류 완벽 가이드

VO, DTO, DAO, Entity에 대한 올바른 이해와 실무 적용법

목차

1. [개요](#)
2. [VO \(Value Object\)](#)
3. [DTO \(Data Transfer Object\)](#)
4. [DAO \(Data Access Object\)](#)
5. [Entity \(개체\)](#)
6. [정리](#)

개요

객체의 여러 종류 중에서도 유명한 **VO, DTO, DAO, Entity**에 관해 살펴볼 것이다.

일반적인 질문들

- **VO**란 어떤 객체인가?
- **DTO**란 어떤 객체일까?
- **DAO**란 어떤 객체일까?
- **Entity**란 어떤 객체일까?

저연차 개발자들의 일반적인 답변

VO: Value Object로 값 객체라는 뜻이다. 쓰기 작업이 불가능한 읽기 전용 객체이다.

DTO: 계층 간 데이터 교환에 사용되는 객체이다. 예시로 DB에 데이터 넣을 때 DTO를 이용한다.

DAO: DB에 접근하는데 사용되는 객체이다.

Entity: JPA의 @Entity이며 테이블에 1:1로 대응되고 각각 구분할 수 있는 식별자를 갖고 있다.

문제점

인터넷에 검색해보면 위와 같은 답변이 마치 정답인 것처럼 알려주는 글을 많이 볼 수 있다. 하지만 안타깝게도 대부분은 충분하지 못하며 **틀린 답변도 존재한다.**

예를 들어 **Entity**는 **JPA의 Entity**라는 설명이 있다. 이에 따르면 Entity는 JPA 없으면 설명할 수 없는 개념이 되어버린다. 하지만 JPA가 없는 프로젝트에서도 Entity라는 개념을 가져다 잘 사용한다.

💡 **핵심:** 개념을 이해하지 않고 외우려고만 할 때 이러한 문제가 발생한다.

😞 시작 전 질문 리스트

```
@Getter
public class UserInfoVo {
    private final long id;
    private final String username;
    private final String password;
    private final String email;
}
```

이게 VO가 맞을까?

🔍 생각해볼 질문들

- VO를 설명하기 위한 키워드로 읽기 전용이라는 특징이 전부인가?
- VO를 쓰는 이유는 무엇이고, 왜 읽기 전용으로 만들려는 걸까?
- 위의 예제는 왜 VO이고 VO라면 왜 그렇고 아니면 왜 아닌가?
- DTO를 설명하라고 하면 대부분 데이터베이스와 연관된 사례를 예시로 드는데 DB를 예시로 들지 않고 설명 가능한가?
- Entity는 JPA의 Entity(@Entity)와 1:1로 대응되는 개념일까?
- 도메인 Entity라는 용어와 DB Entity라는 용어가 있는데 둘의 차이는 무엇일까?

🎨 VO (Value Object : 값 객체)

📄 예제 코드

```
public final class Color {
    public final int r;
    public final int g;
    public final int b;

    public Color(int r, int g, int b) {
        if (r < 0 || r > 255 || g < 0 || g > 255 || b < 0 || b > 255) {
            throw new IllegalArgumentException("RGB should be 0 to 255");
        }

        this.r = r;
        this.g = g;
        this.b = b;
    }
}
```

```
// equals, hashCode 메소드 오버라이드  
}
```

VO의 핵심 개념

Color 클래스가 VO라는 것은 **Color** 클래스로 만들어진 객체를 숫자 1, 2, 3, 4 같은 **값(value)**과 같이 볼 수 있다는 의미이다.

즉, Color는 객체지만 동시에 **값**이라는 소리다. 그래서 **값 객체**라고 부른다.

VO의 3가지 특징

소프트웨어 설계자 입장에서 값은 **불변성**, **동등성**, **자가검증**이라는 특징이 있다. 따라서 이 3가지 특징을 만족할 때 VO라고 부른다.

1 불변성 (Immutability)

개념

불변성이란 말 그대로 변하지 않는다는 의미이다. 값은 변하지 않는다. 예를 들어 숫자 1은 언제나 영원불변하게 항상 숫자 1이다.

장점

변하지 않는다는 개념은 **시스템의 복잡도를 획기적으로 낮출 수 있는 개념**이다. 이 불변성이라는 특징 덕분에 소프트웨어 중 일부를 **예측할 수 있고 신뢰할 수 있게 만들 수 있다**.

구현

```
public class Color {  
    public final int r; // final로 불변성 보장  
    public final int g;  
    public final int b;  
  
    public Color(int r, int g, int b) {  
        this.r = r;  
        this.g = g;  
        this.b = b;  
    }  
}
```

주의사항

"모든 멤버 변수가 final이면 불변이다"라는 명제는 틀렸다!

```
@Getter  
public class FilledColor {  
    public final int r;  
    // ...  
}
```

```

public final int g;
public final int b;
public final Shape shape; // 참조 타입!

public FilledColor(int r, int g, int b, Shape shape) {
    this.r = r;
    this.g = g;
    this.b = b;
    this.shape = shape;
}
}

@Data
class Rectangle extends Shape {
    private int width; // 불변이 아님!
    private int height; // 불변이 아님!

    public Rectangle(int width, int height) {
        this.width = width;
        this.height = height;
    }
}

```

```

Rectangle rectangle = new Rectangle(10, 20);
FilledColor filledColor = new FilledColor(1, 0, 0, rectangle);

// filledColor의 내부 값이 변경돼 불변성이 깨졌다!
filledColor.getShape().setWidth(20);

```

완전한 불변성을 위한 조건

1. 모든 멤버 변수가 불변이어야 한다
2. 모든 함수가 순수함수여야 한다
3. 상속을 금지해야 한다 (final class)

2 동등성 (Equality)

개념

어떤 객체가 값이고 상태가 모두 같다면 같은 객체로 봐야한다.

예제

```

Color green1 = new Color(0, 1, 0);
Color green2 = new Color(0, 1, 0);

System.out.println(green1 == green2); // false를 반환한다

```

이때 `green1 == green2` 는 `true`를 반환해야 할까 아니면 `false`를 반환할까?

VO의 답: 값은 내재된 의미가 같다면 같은 것이다!

🔧 구현 방법

따라서 VO를 만들기 위해 자바에서는 `equals` 나 `hashCode` 를 오버라이딩할 필요가 있다.

```
@Override
public boolean equals(Object obj) {
    if (this == obj) return true;
    if (obj == null || getClass() != obj.getClass()) return false;
    Color color = (Color) obj;
    return r == color.r && g == color.g && b == color.b;
}

@Override
public int hashCode() {
    return Objects.hash(r, g, b);
}
```

🚫 식별자 금지

VO에는 식별자를 넣어서는 안된다. VO는 id 같은 식별자 필드를 멤버 변수로 가지고 있어서는 안된다.

왜냐하면 식별자 정의와 VO 동등성 개념이 서로 충돌하기 때문이다.

3 자가 검증 (Self-Validation)

🧠 개념

자가 검증이란 클래스 스스로 상태가 유효한지 검증할 수 있음을 의미한다. 즉, 유효하지 않은 상태의 객체가 만들어질 수 없다는 것을 의미한다.

🎯 목표

한번 생성된 VO의 멤버 변수에는 이상한 값이 들어 있을 수 없다.

🔧 구현

```
public Color(int r, int g, int b) {
    if (r < 0 || r > 255 || g < 0 || g > 255 || b < 0 || b > 255) {
        throw new IllegalArgumentException("RGB should be 0 to 255");
    }

    this.r = r;
    this.g = g;
}
```

```
this.b = b;
}
```

💡 가치

자가 검증이 완벽한 객체라면 외부에서 이 객체를 사용할 때 상태 검증을 위해 if-else문, try-catch문을 사용하지 않아도 된다.

📦 DTO (Data Transfer Object: 데이터 전송 객체)

🎯 핵심 개념

DTO를 직역하면 데이터 전송에 사용되는 객체를 의미한다.

```
public class UserCreateRequest {
    public String username;
    public String password;
    public String email;
    // ...
}
```

🚀 사용 목적

DTO는 그저 데이터를 하나하나 일일이 나열해서 전달하는 게 불편해서 데이터를 하나로 묶어서 보내려고 만들어진 객체이다.

📋 특징

- 오롯이 데이터를 효과적으로 전달하는데 집중한다
- 그 밖의 능동적인 역할이나 책임을 갖고 있지 않다
- DTO에는 데이터를 읽고 쓰는 것 외에 다른 비즈니스 로직이 들어가서는 안된다

🚫 DTO에 관한 3가지 오해

❌ 오해 1: DTO는 프로세스, 계층 간 데이터 이동에만 사용된다

올바른 이해: DTO의 목적은 데이터를 전달하는 것이다. 따라서 데이터를 전달하고 싶은 상황이라면 어디서든 사용될 수 있다.

❌ 오해 2: DTO는 게터, 세터를 갖고 있다

올바른 이해: 게터, 세터는 내부 데이터를 전달하기 위한 방법 중 하나일 뿐이다.

```
// 의미 없는 캡슐화
public class UserCreateRequest {
    private String username; // private인데
    private String password;
    private String email;

    // 모든 필드에 게터/세터가 있다면 public과 다를 바 없다
    public String getUsername() { return username; }
    public void setUsername(String username) { this.username = username; }
    // ...
}

// 차라리 이렇게
public class UserCreateRequest {
    public String username;
    public String password;
    public String email;
}
```

❌ 오해 3: DTO는 데이터베이스에 데이터를 저장하는 데 사용되는 객체이다

올바른 이해: DTO는 말 그대로 데이터를 전송하기 위한 객체이다. DB뿐만 아니라 API 통신, 객체 간 데이터 교환 등 모든 곳에서 사용될 수 있다.

🔧 @Data 어노테이션에 대한 고찰

```
@Data
public class UserCreateRequest {
    private String username;
    private String password;
    private String email;
}
```

주의사항: @Data 를 자주 사용하는 것은 자연스러운 객체지향 코드를 만드는데 방해가 된다. 이를 사용하는 순간, '이 객체는 말 그대로 객체가 아니라 그저 데이터 덩어리'라고 선언하는 것과 같기 때문이다.

📁 DAO (Data Access Object: 데이터 접근 객체)

🎯 정의

DAO는 데이터베이스 접근과 관련된 역할을 지닌 객체를 가리키는 용어이다.

📋 주요 역할

- DB와의 연결을 관리

- DB에 연결해 데이터에 대한 CRUD 연산을 수행
- 보안 취약성을 고려한 쿼리 작성

핵심 목적

DAO가 만들어진 목적은 **도메인 로직**과 **데이터베이스 연결 로직**을 분리하기 위해서이다.

스프링과의 관계

스프링 개발자에게 친숙한 **Repository**와 같은 개념이라 보면 된다.

설계 철학

- 데이터베이스와의 상호작용은 데이터를 저장하고 검색하는 **기술에 불과하다**
- 실제 애플리케이션의 핵심은 **요구사항을 해결하는 비즈니스 로직**이고 **도메인**이다
- 따라서 이 둘을 분리하는 것이 중요하다

Entity (개체)

가장 큰 오해

| Entity = JPA의 @Entity ❌

이런 식의 접근은 틀렸다. 왜냐하면:

- Entity는 **JPA에서 만들어진 용어가 아니다**
- 이러한 개념은 **JPA가 없던 시절부터 존재했던 개념**이다
- JPA의 Entity는 Entity라는 개념을 표현하기 위한 **수단 중 하나일 뿐**이다

3가지 Entity 유형

1 도메인 Entity

도메인: 해결하고자 하는 문제 영역 (비즈니스 영역)

```
// 은행 도메인의 예시
public class Account {
    private final AccountId id;           // 식별자
    private Money balance;                 // 상태

    public void withdraw(Money amount) { // 비즈니스 로직
        if (balance.isLessThan(amount)) {
            throw new InsufficientBalanceException();
        }
        this.balance = balance.subtract(amount);
    }
}
```

```
}  
}
```

특징:

- 식별 가능한 식별자를 갖는다
- 비즈니스 로직을 갖는다
- 생애주기를 갖는다

2 DB Entity

관계형 데이터베이스 분야에서 어떤 유무형의 객체를 표현하는데 사용했던 용어이다.

3 JPA Entity

```
@Entity  
@Table(name = "accounts")  
public class AccountEntity {  
    @Id  
    private Long id;  
  
    @Column(name = "balance")  
    private BigDecimal balance;  
  
    // getter, setter...  
}
```

정확한 이해:

- JPA Entity는 **관계형 데이터베이스의 Entity**를 지칭하는 것이다
- JPA의 @Entity 어노테이션이 적용된 객체는 "**영속성 객체(PO: Persistent Object)**"이다
- JPA의 @Entity 어노테이션은 **영속성 객체를 만들기 위한 도구일 뿐**이다

Entity의 본질적 의미

Entity: 데이터로 표현하려는 **유무형의 대상**

프로그래밍 언어와 데이터베이스 분야에서는 표현하고 싶은 유무형의 자산 정보를 지칭하는 데 **개체 (Entity)**라는 용어를 사용하기로 했다.

- **객체지향 진영:** Entity를 표현하는데 **클래스**를 사용
- **데이터베이스 진영:** 테이블을 사용해 데이터를 표현

발전 과정

1. **초기:** 각자의 세계에서 잘 적용돼 나름의 세상을 구축
2. **협업 필요:** 실세계의 서비스를 만들기 위해 양쪽이 협업 필요

3. 매핑 라이브러리: MyBatis 같은 라이브러리 등장
4. ORM 등장: JPA, Hibernate 같은 **Object-Relational Mapping** 솔루션 등장

정리

구분	설명	특징
Entity	데이터로 표현하려는 유무형의 대상	범용적 개념
도메인 Entity	소프트웨어 개발 분야에서 말하는 Entity	식별자 + 비즈니스 로직
DB Entity	데이터베이스 분야에서 데이터로 표현하려는 대상	테이블과 연관
JPA Entity	관계형 데이터베이스의 Entity를 지칭	영속성 객체

💡 **핵심:** 도메인 Entity와 DB Entity는 다르다!

정리

핵심 가치들

지금까지 배운 가치는 다음과 같다:

- 불변성 (Immutability)
- 예측 가능성 (Predictability)
- 역할의 분리 (Separation of Concerns)
- 일관성 (Consistency)

마무리 철학

객체의 타입을 분류하는 데 치중하기보다 위와 같은 가치들을 프로젝트에 **지속적으로 적용**해 보길 바란다.

사실 이러한 개념을 외우고 엄격한 기준을 적용하는 것보단 **각 개념이 만들어진 이유와 목적을 생각하는 것**이 바람직하다.

그저 이름을 붙이는 것이 아닌 **실제로 각 개념을 이해하고 적용하는 과정에서** 소프트웨어 개발 역량이 향상 될 것이다.